

Examen Réparti 1 PSCR Master 1 Informatique Nov 2022

UE 4I400

Année 2022-2023

2 heures – **Tout document papier autorisé**
Tout appareil de communication électronique interdit (téléphones...)

Introduction

- Le barème est sur 20,5 points et est donné à titre indicatif.
- Dans le code C++ demandé vous prendrez le temps d'assurer que la compilation fonctionne.
- On vous fournit une archive contenant les sources qu'il faudra modifier. On vous laisse libre d'utiliser l'éditeur de sources ou IDE que vous souhaitez. On vous fournit un Makefile trivial (cf son contenu pour le nom des cibles). Vous pouvez modifier l'arborescence des sources et les réorganiser tant que les fichiers de réponse conservent leur nom, et restent dans un sous dossier de `~/exam/`. Si vous utilisez eclipse, démarrer un workspace dans ce dossier, y créer des projets C++ et importer les sources peut fonctionner.
- A la fin de la séance, fermez simplement votre session. On ira par script récupérer les fichiers dans ce fameux dossier `~/exam/`. Assurez vous donc de bien suivre ces instructions.
- Vous n'avez pas un accès complet à internet, mais si vous aurez accès au site <https://cppreference.com>

Le sujet est composé de trois exercices indépendants qu'on pourra traiter dans l'ordre qu'on souhaite.

1 ResultatCalcul (10 points)

1.1 Classe ResultatCalcul

La classe `ResultatCalcul` héberge un résultat (un `std::string` pour s'épargner une version générique) qui doit être calculé par un thread esclave et accédé par un thread maître. Il permet au maître d'attendre la complétion du travail du ou des esclaves.

On propose l'API suivante :

- `const string & getResult() const` : Rend la valeur stockée dès qu'elle est disponible. L'invocation bloque l'appelant si nécessaire jusqu'à ce que la valeur devienne disponible.
- `bool isSet() const` : est vrai si et seulement si la valeur a déjà été affectée, i.e. `getResult` n'est plus bloquant.
- `void setResult(const string & res)` : permet de positionner la valeur stockée, débloque les threads éventuellement bloqués sur `getResult`. On ne doit invoquer `setResult` qu'une seule fois sur une instance donnée de `ResultatCalcul`, si on l'invoque de nouveau cela n'a aucun effet.

Question 1. (3,5 points) Implanter cette classe avec son constructeur par défaut (le résultat n'est pas encore disponible) et si nécessaire un destructeur. Sans que ce soit imposé, on propose de rédiger directement dans "ResultatCalcul.hh" sans créer de .cpp.

ResultatCalcul.h

```

1  #pragma once
2
3  #include <mutex>
4  #include <condition_variable>
5
6
7  class ResultatCalcul {
8      std::string valeur;
9      mutable std::mutex m;
10     mutable std::condition_variable cv;
11     bool isSet_;
12 public:
13     ResultatCalcul() : isSet_(false) {}
14     const std::string & getResult () const {
15         std::unique_lock<std::mutex> l(m);
16         while ( ! isSet_ ) {
17             cv.wait(l);
18         }
19         return valeur;
20     }
21     bool isSet () const {
22         std::unique_lock<std::mutex> l(m);
23         return isSet_;
24     }
25     void setResult (const std::string & res) {
26         std::unique_lock<std::mutex> l(m);
27         if (! isSet_) {
28             valeur = res;
29             isSet_ = true;
30             cv.notify_all();
31         }
32     }
33 };
34

```

Barème :

- 10 % ctor, initialisation, syntaxe générale de la classe
- 20 % attributs : mutex, condition, valeur, bool
- 10 % le mutex et la condition sont mutable
- 20 % protection de toutes les opérations (unique lock)
- 20 % notification dans set
- 20 % wait sur une condition dans get

On attend un destructeur si on a fait le choix de stocker la string par pointeur. Dans cette version le pointeur vaut *nullptr* initialement, ce qui évite d'avoir à ajouter un bool (on teste *value == nullptr*). Il faut s'assurer que dans setResult qu'on copie la string dans ce cas pour avoir la responsabilité de la mémoire pointée. Dans la version du corrigé le dtor n'est pas nécessaire. Donc si on a fait pointeur mais pas de dtor, -10 % de pénalité.

Attention aux deadlocks si on utilise un mutex mais pas un recursive_mutex, et qu'on invoque *isSet()* au lieu de consulter simplement l'attribut bool (*isSet_* dans le corrigé). On est déjà sous lock donc une double acquisition => deadlock trivial avec un mutex simple. -30 % si la solution introduit un deadlock.

1.2 Utilisation de la classe

On souhaite écrire un programme qui :

- crée un vecteur de **ResultatCalcul** de taille N (lu comme premier argument de la ligne de commande)
- construit N threads esclave, chaque esclave va calculer un des résultats : la valeur calculée sera simplement "slaveXX" où XX est son *numéro d'ordre de création*.
- affiche le contenu du vecteur, une valeur par ligne, quand il est calculé
- join les threads lancés
- se termine proprement, sans fuite mémoire etc...

Question 2. (2,5 points) Proposez un programme C++ (un **main**) qui a ce comportement et qui maximise le parallélisme. On interdit l'utilisation de variables globales.

On répondra dans un fichier : "mainQ12.cpp"

```

1  #pragma once
2
3  #include <vector>
4  #include "ResultatCalcul.h"
5
6
7  void slave (int indice, ResultatCalcul & rc) {
8      rc.setResult("Slave" + std::to_string(indice) );
9  }
10
11 #define N 10
12
13 int main () {
14
15     std::vector<ResultatCalcul> res (N);
16     std::vector<std::thread> threads;
17     threads.reserve(N);
18
19     for (int i=0 ; i < N ; i++) {
20         threads.emplace_back(slave,i,std::ref(res[i]));
21     }
22
23     for (ResultatCalcul & r : res) {
24         std::cout << res.getValue() << std::endl;
25     }
26
27     for (auto & t : threads) {
28         t.join();
29     }
30
31 }
```

Barème : 70 sur le main :

- 10 % création et initialisation du vecteur de ResultatCalcul
- 30 % appel au ctor de thread, dans une boucle qui stocke les thread (10), avec des paramètres OK, en particulier le passage de l'indice (10), et du résultat calcul par référence (qualifiée par `std::ref`) ou pointeur (10).
- 20 % getResult
- 10 % join

30 sur la fonction passé au thread : 10 la signature, 10 on fait setResult, 10 on construit bien une string concaténée, e.g. avec to_string ou un stringstream.

Quand N est grand, on constate que créer trop de threads n'est pas rentable. On souhaite au contraire ne construire que K threads esclave (au lieu de N), en supposant K petit devant N . Chaque esclave dans cette version doit traiter N/K tâches (on ignore l'arrondi, on suppose que N est multiple de K).

On fait donc un découpage par lots de taille égale des résultats à calculer, chaque lot ayant une taille N/K .

Question 3. (2 points) Modifiez votre programme pour avoir ce comportement. K sera lu comme le deuxième argument de la ligne de commande.

On répondra dans un fichier : "mainQ13.cpp"

On conseille de partir d'une copie du main de la question précédente.

mainCalcul2.cpp

```

1  #pragma once
2
3  #include <vector>
4  #include "ResultatCalcul.h"
5
6  #define N 100
7  #define K 10
8
9  void slave (int indice, std::vector<ResultatCalcul> & vec) {
10     int deb = indice * (N/K);
11     int fin = deb + (N/K);
12     for (int cible = deb ; cible < fin ; cible++) {
13         vec[cible].setResult("Slave" + std::to_string(indice) );
14     }
15 }
16
17 int main () {
18
19     std::vector<ResultatCalcul> res (N);
20     std::vector<std::thread> threads;
21     threads.reserve(K);
22
23     for (int i=0 ; i < K ; i++) {
24         threads.emplace_back(slave,i,std::ref(res[i]));
25     }
26
27     for (ResultatCalcul & r : res) {
28         std::cout << res.getValue() << std::endl;
29     }
30
31     for (auto & t : threads) {
32         t.join();
33     }
34
35 }
```

BarèmeTODO :

- 20 % signature adaptée pour passer le vecteur/une plage de valeurs, SANS COPIE

- 20 % on place des choix d'indices corrects, i.e. N/K tâches par thread. Ca peut se faire dans le main plutôt, selon la signature choisie.
- 40 % on a clairement une boucle dans la fonction passée au ctor de thread
- 20 % mise à jour congruente avec ces modifications du main

1.3 Comparaison à un Pool

On voudrait comparer le comportement de votre programme à une version qui utiliserait un pool de threads, comme élaboré en TD/TME, et construirait un **Job** pour chaque résultat à calculer. (NB : on ne demande pas le code de cette version !)

Question 4. (2 points) Expliquez en quoi la solution avec un Pool démarré avec K threads diffère de la solution proposée en Q1.3 si on suppose que le calcul de chaque résultat a une durée fixe (homogène) ou au contraire que le calcul d'un résultat prend une durée aléatoire difficile à prédire (mais conséquente pour chaque calcul). On s'intéresse à l'utilisation des ressources de la machine en temps total de calcul sur une machine multi-core.

On répondra dans le fichier "question14.txt".

Ressources : si la charge est homogène définir a priori les paquets est OK (stratégie implantée dans cet énoncé). Dans le cas contraire il faut préférer la version Pool de thread, car dans la version actuelle certains threads pourraient se retrouver idle alors qu'il reste du travail à effectuer.

BarèmeTODO :

- 50 % si les tâches ont la même taille => change pas grand chose
- 50 % sinon, le Pool doit être considéré meilleur

2 Arbre, pointeurs, gestion mémoire en C++(6,5 points)

On considère un arbre binaire de recherche, chaque noeud de l'arbre (on utilisera une seule classe) porte deux pointeurs vers les sous-arbres gauche et droit (qui peuvent être `nullptr` désignant un sous-arbre vide), ainsi qu'une string. Le sous-arbre gauche (resp. droit) contient des string lexico-graphiquement inférieures strictement (respectivement supérieures strictement) à la string du noeud courant. On a un ensemble au sens mathématique, un seul noeud de l'arbre porte une string donnée.

Question 1. Donnez le code de la classe **Node** :

- (0,5 pt) ses attributs left, right et value.
- (0,5 pt) un constructeur à un seul argument (la string) qui initialise les sous arbres gauche et droit à vide.
- (2 pt) `bool insert(const string & s)` : essaie d'insérer la string *s* et rend vrai si une insertion a lieu, et faux sinon. Soit le noeud courant porte déjà la string à insérer, soit il faut réaliser une récursion sur le sous arbre adapté (en comparant *s* à la valeur stockée). Si la récursion atteint un sous arbre vide on crée un nouveau noeud qu'on accroche dans l'arbre.
- (1 pt) un constructeur par copie, qui suit la signature standard `Node(const Node &)` et copie tout l'arbre. On ne demande pas l'opérateur d'affectation qui serait similaire.
- (1 pt) un destructeur qui doit désallouer tout l'arbre dont le noeud courant est la racine.
- (1,5 pt) un opérateur d'affichage standard C++, qui affiche tous les mots contenus dans l'arbre dans l'ordre alphabétique (avec un retour à la ligne après chaque mot).

On répondra dans le fichier "Node.hh". Le "mainQ21.cpp" fourni est censé tourner sans erreur (e.g. de valgrind).

Node.h

```

1  #pragma once
2
3  #include <string>
4
5  class Node {
6      Node * left;
7      Node * right;
8      std::string s;
9  public :
10     Node (const std::string & s):left(nullptr),right(nullptr),s(s){}
11     bool insert (const std::string & word) {
12         if (word == s) {
13             return false;
14         } else if ( word < s ) {
15             if (left == nullptr) {
16                 left = new Node(word);
17                 return true;
18             } else {
19                 return left->insert(word);
20             }
21         } else {
22             if (right == nullptr) {
23                 right = new Node(word);
24                 return true;
25             } else {
26                 return right->insert(word);
27             }
28         }
29     }
30     Node (const Node & other) {
31         if (other.left) left = new Node(*other.left); else left = nullptr;
32         if (other.right) right = new Node(*other.right); else right = nullptr;
33         s = other.s;
34     }
35     ~Node() {
36         delete left;
37         delete right;
38     }
39     ostream & operator<<(ostream & os, const Node & n) {
40         if (n.left) os << *n.left;
41         os << value << std::endl;
42         if (n.right) os << *n.right;
43     }
44 };

```

BarèmeTODO :

- 10 % attributs, structure de la classe
- 10 % ctor qui pose des valeurs pour l et r
- 40 % insert : 10 % cas terminal false, 10% cas new+true, 10 % recursion; 10% cas right symétrique de left (même s'il est faux).
- 20 % ctor par copie
- 20 % dtor
- 30 % operateur d'affichage

NB: Barème sur 130% à saisir tel quel dans la feuille.

3 Arbres de processus (4 points)

On considère le code suivant dans le fichier mainQ31.cpp.

fork.cpp

```

1  #include <iostream>
2  #include <sys/types.h>
3  #include <unistd.h>
4
5  // a faire varier
6  #define N 4
7
8  int main () {
9      int i = -1;
10     int j = -1;
11     for (i=0; i < N; ++i) {
12         if (fork() == 0) {
13             for (j=i+1; j < N; ++j) {
14                 if (fork() != 0) {
15                     break;
16                 }
17             }
18             break;
19         }
20     }
21     std::cout << "Processus " << getpid() << std::endl;
22     return 0;
23 }
```

Question 1. (1,5 points) Donnez le nombre total (sans compter le main) de processus engendrés pour $N=1$, $N=2$, $N=3$, $N=4$.

On répondra dans le fichier question31.txt

$N=1 \Rightarrow 1$ fils
 $N=2 \Rightarrow 1 + 1$ fils qui crée 1 fils $\Rightarrow 3$
 $N=3 \Rightarrow 1$ qui crée 2 et un qui crée 1 + 1 $\Rightarrow 6$
 $N=4 \Rightarrow 1$ qui crée 3, 1 qui crée 2, 1 qui crée 1 $\Rightarrow 1 + 4 + 3 + 2 \Rightarrow 10$
 Barème :
 • 25 % chaque bonne réponse

Question 2. (2,5 points) Tout en maximisant le parallélisme, ajouter les instructions `wait` utiles juste avant le `return 0`; pour garantir que le main ne se termine qu'après que l'ensemble de ses descendants soient terminés. NB: On demande d'éviter d'invoquer `wait` plus de fois que l'on a créé de processus fils. On recommande soit de s'appuyer sur les valeurs de i et j pour décider du comportement à adopter après les boucles, soit d'introduire une nouvelle variable comptabilisant le nombre de fils.

On répondra en modifiant le fichier fourni "mainQ31.cpp"

forkcor.h

```

1  #include <iostream>
2  #include <sys/types.h>
3  #include <unistd.h>
```

```

4
5 int main () {
6     int i = -1;
7     int j = -1;
8     for (i=0; i < N; ++i) {
9         if (! fork()) {
10             for (j=i; j < N; ++j) {
11                 if (fork()) {
12                     break;
13                 }
14             }
15             break;
16         }
17     }
18     std::cout << "Processus " << getpid() << std::endl;
19
20     if (i == N) {
21         // main => N fils total
22         for (int i=0; i < N;i++) {
23             int status;
24             wait(&status);
25         }
26     } else {
27         // j == N => dernier descendant de la chaine
28         // tous les autres, un seul fils
29         if (j < N) {
30             int s;
31             wait(&s);
32         }
33     }
34
35     return 0;
36 }

```

Barème : 90 % sur les trois alternatives à considérer. On cherche trois cas : la façon de tester quel cas c'est vaut 10% (conditions précises dans les if), l'identification du cas vaut 20%.

- 30 % le cas N fils
- 30 % le cas 1 fils
- 30 % le cas 0 fils
- 10 % syntaxe du wait est ok

NB: on gagne les 90% si on a introduit une variable nfils qu'on a correctement gérée, i.e. qu'on a reset après un fork si on est le fils de façon systématique.